

# Operating Systems – Exam Notes

## Contents

|          |                                                    |           |
|----------|----------------------------------------------------|-----------|
| <b>1</b> | <b>Process States</b>                              | <b>3</b>  |
| 1.1      | The Five Classical States . . . . .                | 3         |
| 1.2      | Meaning of Each State . . . . .                    | 3         |
| 1.3      | Important State Transitions . . . . .              | 3         |
| <b>2</b> | <b>Scheduling</b>                                  | <b>5</b>  |
| 2.1      | Scheduling Criteria . . . . .                      | 5         |
| 2.2      | SJF (Shortest Job First) . . . . .                 | 5         |
| 2.3      | STCF (Shortest Time to Completion First) . . . . . | 5         |
| 2.4      | Difference Between SJF and STCF . . . . .          | 5         |
| 2.5      | Scheduling after Deadlines . . . . .               | 6         |
| <b>3</b> | <b>Deadlocks</b>                                   | <b>7</b>  |
| 3.1      | Necessary Conditions . . . . .                     | 7         |
| 3.2      | Example . . . . .                                  | 7         |
| 3.3      | Preventing Deadlocks . . . . .                     | 7         |
| <b>4</b> | <b>Memory Management</b>                           | <b>9</b>  |
| 4.1      | Address Translation . . . . .                      | 9         |
| <b>5</b> | <b>Paging</b>                                      | <b>10</b> |
| 5.1      | Address Structure . . . . .                        | 10        |
| 5.2      | Page Table . . . . .                               | 10        |
| 5.3      | Valid Bit . . . . .                                | 10        |
| 5.4      | Demand Paging . . . . .                            | 10        |
| <b>6</b> | <b>Segmentation</b>                                | <b>12</b> |
| 6.1      | Translation . . . . .                              | 12        |
| 6.2      | Paging vs Segmentation . . . . .                   | 12        |
| <b>7</b> | <b>Locality of Reference</b>                       | <b>13</b> |
| 7.1      | Temporal Locality . . . . .                        | 13        |
| 7.2      | Spatial Locality . . . . .                         | 13        |
| <b>8</b> | <b>CPU Cache</b>                                   | <b>14</b> |
| 8.1      | Cache Terminology . . . . .                        | 14        |
| 8.2      | Cache Organizations . . . . .                      | 14        |
| <b>9</b> | <b>Page Replacement Policies</b>                   | <b>16</b> |
| 9.1      | FIFO . . . . .                                     | 16        |
| 9.2      | LRU . . . . .                                      | 16        |
| 9.3      | Clock Algorithm . . . . .                          | 16        |
| 9.4      | NRU (Not Recently Used) . . . . .                  | 17        |

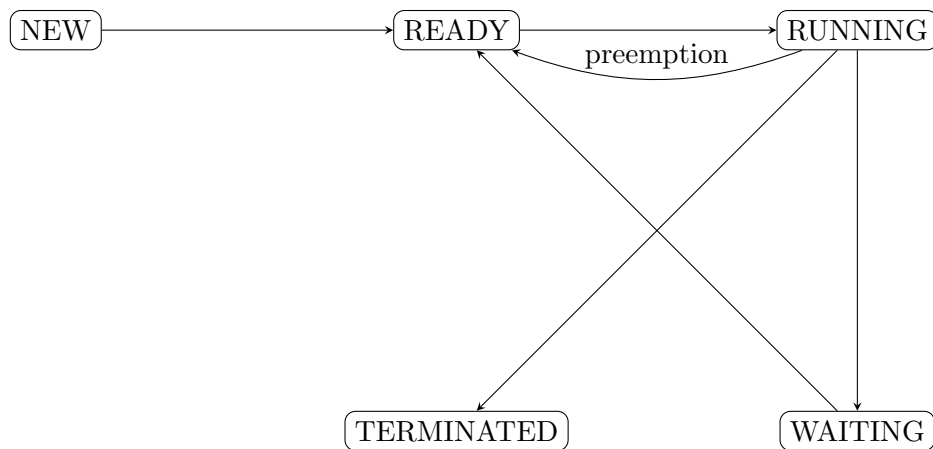
|                                           |           |
|-------------------------------------------|-----------|
| <b>10 Free Space Management</b>           | <b>18</b> |
| 10.1 First Fit . . . . .                  | 18        |
| 10.2 Best Fit . . . . .                   | 18        |
| 10.3 Worst Fit . . . . .                  | 18        |
| 10.4 Solved Example . . . . .             | 18        |
| <b>11 File Systems</b>                    | <b>20</b> |
| 11.1 Linux Inode . . . . .                | 20        |
| 11.2 Inode Structure . . . . .            | 20        |
| 11.3 Maximum File Size . . . . .          | 21        |
| 11.4 Hard Links . . . . .                 | 21        |
| 11.5 Symbolic Links . . . . .             | 22        |
| 11.6 Hard Link vs Symbolic Link . . . . . | 22        |
| 11.7 Superblock . . . . .                 | 22        |
| <b>12 RAID</b>                            | <b>23</b> |
| 12.1 RAID 0 . . . . .                     | 23        |
| 12.2 RAID 1 . . . . .                     | 23        |
| 12.3 RAID 5 . . . . .                     | 23        |
| 12.4 RAID 6 . . . . .                     | 23        |
| 12.5 RAID 10 . . . . .                    | 24        |
| <b>13 Miscellaneous</b>                   | <b>25</b> |
| 13.1 Zombie Process . . . . .             | 25        |
| 13.2 Process vs Thread . . . . .          | 25        |
| 13.3 Runtime of open/read . . . . .       | 25        |
| <b>14 Exam Cheat Sheet</b>                | <b>26</b> |

# 1 Process States

Every process changes its state during execution.

The operating system scheduler moves the processes between these states.

## 1.1 The Five Classical States



## 1.2 Meaning of Each State

| State                | Meaning                                                                                                                                                                  |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| NEW                  | Process has just been created.                                                                                                                                           |
| READY                | The process has everything it needs except the CPU. It is waiting in the ready queue.                                                                                    |
| RUNNING              | Currently executing instructions on the CPU.                                                                                                                             |
| WAITING<br>(BLOCKED) | Waiting for an event: <ul style="list-style-type: none"><li>• keyboard input</li><li>• disk access</li><li>• pipe</li><li>• semaphore</li><li>• network packet</li></ul> |
| TERMINATED           | Execution finished. Resources can be reclaimed.                                                                                                                          |

## 1.3 Important State Transitions

READY → RUNNING

The scheduler chooses the process.

RUNNING → WAITING

The process requests I/O or waits for an event.

WAITING → READY

The event finishes.

RUNNING → READY

Occurs because of:

- timer interrupt
- higher priority process

- preemption

This transition is one of the favorite exam questions.

### Things to Remember

- READY means "waiting for CPU".
- WAITING means "waiting for an event".
- A blocked process CANNOT execute.
- Only one process can be RUNNING on one CPU core.

### Possible Exam Questions

What event can move a process from RUNNING to READY?  
Explain the difference between READY and WAITING.  
Draw the process state diagram.

## 2 Scheduling

Scheduling determines which READY process receives the CPU.

### 2.1 Scheduling Criteria

A scheduler tries to maximize:

- CPU utilization
- Throughput
- Fairness
- Low waiting time
- Low turnaround time
- Low response time

### 2.2 SJF (Shortest Job First)

Always executes the process with the smallest execution time.

Advantages:

- minimal average waiting time
- efficient for batch systems

Disadvantages:

- execution time must be known
- long jobs may starve

### 2.3 STCF (Shortest Time to Completion First)

Also called:

Shortest Remaining Time First

It is the preemptive version of SJF.

Whenever a new shorter process arrives:

*Current Process*  $\rightarrow$  *READY*

The new process immediately starts running.

### 2.4 Difference Between SJF and STCF

| SJF                       | STCF                          |
|---------------------------|-------------------------------|
| Non-preemptive            | Preemptive                    |
| Current job finishes      | Can interrupt current job     |
| Uses total execution time | Uses remaining execution time |

This is probably the most common scheduling question.

## 2.5 Scheduling after Deadlines

Some systems schedule processes according to deadlines.

Examples:

- Embedded systems
- Medical devices
- Airplanes
- Automotive software

Missing a deadline may cause system failure.

### Things to Remember

SJF

*Shortest Job*

STCF

*Shortest Remaining Time*

STCF may interrupt another process.

### Possible Exam Questions

Explain the difference between SJF and STCF.  
Give one advantage of STCF.  
Can SJF starve long jobs?

### 3 Deadlocks

A deadlock appears when processes wait forever for one another.

#### 3.1 Necessary Conditions

All four conditions must hold.

1. Mutual Exclusion

Only one thread may own a resource.

2. Hold and Wait

A process holds one resource while requesting another.

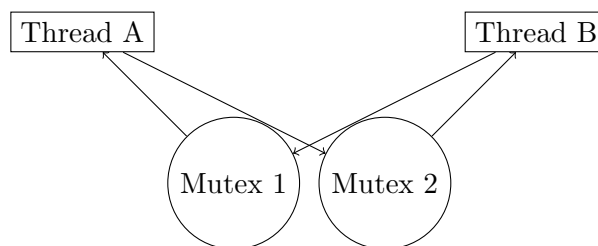
3. No Preemption

Resources cannot be forcibly taken.

4. Circular Wait

A cycle exists.

#### 3.2 Example



Nobody can continue.

#### 3.3 Preventing Deadlocks

The most common solution is:

Always lock mutexes in the SAME ORDER.

Correct:

```
lock(A);
```

```
lock(B);
```

```
/* critical section */
```

```
unlock(B);
```

```
unlock(A);
```

Never sometimes use

$$A \rightarrow B$$

and elsewhere

$$B \rightarrow A$$

## Exam Answer

Question:

What can a programmer do to prevent deadlocks?

Correct answer:

- lock mutexes in a fixed global order
- avoid circular wait
- release resources quickly
- use trylock when appropriate

## Things to Remember

Deadlock

=

Processes wait forever.

Necessary conditions:

1. Mutual Exclusion
2. Hold and Wait
3. No Preemption
4. Circular Wait

## Possible Exam Questions

What are the four necessary conditions of a deadlock?

How can a programmer prevent deadlocks?

Why does locking mutexes in a fixed order work?

## 4 Memory Management

Memory management is responsible for allocating, protecting and translating memory addresses for processes.

The operating system must:

- Allocate memory to processes
- Protect one process from another
- Translate logical addresses into physical addresses
- Reclaim memory after a process terminates

Memory can be allocated using:

- Fixed partitions
- Variable partitions
- Paging
- Segmentation

### 4.1 Address Translation

Programs generate **logical (virtual) addresses**.

The Memory Management Unit (MMU) translates them into **physical addresses**.

#### Absolute Address Translation

For fixed partitions:

$$\text{PhysicalAddress} = \text{Base} + \text{LogicalAddress}$$

where

- Base = beginning of the allocated partition
- LogicalAddress = address generated by the CPU

#### Example

Base = 1000

Logical Address = 25

$$\text{Physical} = 1000 + 25 = 1025$$

#### Diagram



#### Possible Exam Questions

How is address calculation performed in absolute fixed partition allocation?  
Give the translation formula.  
Compute the physical address if:  
Base = 2048  
Logical Address = 150

## 5 Paging

Paging divides memory into equal-sized blocks.

- Process memory = Pages
- RAM = Frames

Each page may be loaded into any free frame.  
This eliminates external fragmentation.

### 5.1 Address Structure

A logical address contains:

|                          |
|--------------------------|
| $PageNumber \mid Offset$ |
|--------------------------|

The page number indexes the page table.  
The offset remains unchanged.

### 5.2 Page Table

Each process owns its own page table.

A page table entry stores:

- Frame number
- Valid bit
- Dirty bit
- Access rights

### 5.3 Valid Bit

- Valid = page is currently in RAM
- Invalid = page is not loaded

If an invalid page is accessed:

|            |
|------------|
| Page Fault |
|------------|

occurs.

### 5.4 Demand Paging

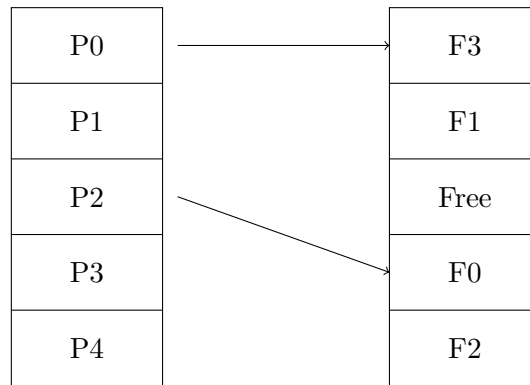
Pages are NOT loaded immediately.

Instead,  
pages are loaded only when first accessed.  
Advantages:

- Less RAM usage
- Faster program startup

Disadvantages:  
First access produces a page fault.

## Diagram



## Things to Remember

Paging removes

External Fragmentation

but still suffers from

Internal Fragmentation

## Possible Exam Questions

When should pages be loaded into memory?  
What happens during a page fault?  
What information is stored in a page table?  
Explain the role of the valid bit.

## 6 Segmentation

Segmentation divides memory into logical components.

Examples:

- Code
- Stack
- Heap
- Global variables

Each segment has

- Base address
- Limit (size)

### 6.1 Translation

Logical address:

|                         |
|-------------------------|
| $SegmentNumber, Offset$ |
|-------------------------|

Physical address:

$$Physical = Base + Offset$$

only if

$$Offset < Limit$$

Otherwise:

Segmentation Fault.

### 6.2 Paging vs Segmentation

|                        | Paging     | Segmentation       |
|------------------------|------------|--------------------|
| Unit                   | Fixed size | Variable size      |
| Visible to programmer  | No         | Yes                |
| External fragmentation | No         | Yes                |
| Internal fragmentation | Yes        | No                 |
| Protection             | Per page   | Per logical module |

### Possible Exam Questions

Compare paging and segmentation.  
Give one advantage of segmentation.  
Give one disadvantage of segmentation.

## 7 Locality of Reference

Most programs repeatedly access nearby memory locations.

This observation is called the **Principle of Locality**.

### 7.1 Temporal Locality

If a memory location is accessed now,  
it is likely to be accessed again soon.

Example:

```
for ( i=0; i < 1000; i++)  
    sum += a[0];
```

The same variable is reused.

### 7.2 Spatial Locality

If one memory location is accessed,  
nearby locations are likely to be accessed next.

Example:

```
for ( i=0; i < n; i++)  
    sum += a[i];
```

Array elements are consecutive in memory.

### Why is Locality Important?

Locality is the reason why:

- caches work
- paging works efficiently
- prefetching improves performance

### Things to Remember

Temporal locality

=

Reuse the same memory.

Spatial locality

=

Access nearby memory.

### Possible Exam Questions

Explain the principle of locality.  
Differentiate temporal and spatial locality.  
Why is locality important for paging?

## 8 CPU Cache

A cache is a small, very fast memory located between the CPU and RAM.

Its purpose is to reduce the average memory access time.

$$\text{CPU} \longleftrightarrow \text{Cache} \longleftrightarrow \text{RAM}$$

The cache stores recently accessed memory blocks.

Because of the principle of locality, future accesses are likely to use the same or nearby data.

### 8.1 Cache Terminology

- Cache Hit

The requested data is found in the cache.

Access is very fast.

- Cache Miss

The requested data is NOT in the cache.

The CPU must fetch it from RAM.

### 8.2 Cache Organizations

There are three common cache organizations.

#### Direct-Mapped Cache

Each memory block has exactly one cache position.

Advantages:

- Very simple
- Very fast
- Cheap hardware

Disadvantages:

- Many conflict misses

#### Fully Associative Cache

Any memory block may be stored anywhere.

Advantages:

- Few conflict misses

Disadvantages:

- Expensive hardware
- Slower lookup

## Set Associative Cache

The cache is divided into sets.

Each block may be placed inside one set.

Advantages:

- Good compromise
- Fewer conflicts than Direct Mapping
- Cheaper than Fully Associative

## Comparison

| Type              | Speed     | Conflict Misses |
|-------------------|-----------|-----------------|
| Direct            | Very High | High            |
| Set Associative   | High      | Medium          |
| Fully Associative | Lower     | Very Low        |

## Possible Exam Questions

Give one advantage and one disadvantage of Set Associative Cache compared to Direct-Mapped Cache.

What is a Cache Hit?

What is a Cache Miss?

## 9 Page Replacement Policies

When RAM is full and a page fault occurs, the operating system must choose a page to remove. This decision is made by a replacement policy.

### 9.1 FIFO

FIFO removes the oldest page.

Example:

Pages loaded:

$$A \rightarrow B \rightarrow C \rightarrow D$$

FIFO removes

$A$

Advantages:

- Very simple

Disadvantages:

- Can remove frequently used pages
- Belady's anomaly may occur

### 9.2 LRU

Least Recently Used.

Removes the page that has not been accessed for the longest time.

Advantages:

- Usually better than FIFO

Disadvantages:

- More expensive to implement

### 9.3 Clock Algorithm

Clock is an approximation of LRU.

Each page has one reference bit.

Algorithm:

- If reference bit = 0  
Replace page.
- If reference bit = 1  
Set it to 0 and continue.

## 9.4 NRU (Not Recently Used)

Each page has two bits:

- Reference bit (R)
- Modified bit (M)

Pages are classified into four classes.

| Class | R | M |
|-------|---|---|
| 0     | 0 | 0 |
| 1     | 0 | 1 |
| 2     | 1 | 0 |
| 3     | 1 | 1 |

The operating system always selects the lowest non-empty class.  
Therefore,

*Class 0*

has the highest priority for replacement.

### Things to Remember

FIFO

Oldest page.

LRU

Least recently used.

Clock

Approximate LRU.

NRU

Choose the lowest class.

### Possible Exam Questions

What page is selected first by NRU?

Compare FIFO and LRU.

Why is Clock considered an approximation of LRU?

## 10 Free Space Management

Dynamic memory allocation must choose a free block large enough to satisfy each allocation request.

Common algorithms:

- First Fit
- Best Fit
- Worst Fit

### 10.1 First Fit

Select the first free block that is large enough.

Advantages:

- Very fast

Disadvantages:

- Fragmentation near the beginning of memory

### 10.2 Best Fit

Choose the smallest block that can satisfy the request.

Advantages:

- Wastes less memory initially

Disadvantages:

- Creates many tiny unusable fragments
- Slow search

### 10.3 Worst Fit

Choose the largest free block.

Advantages:

- Leaves relatively large remaining free blocks

Disadvantages:

- Large blocks disappear quickly
- Often wastes memory

### 10.4 Solved Example

Free list:

32      44      98

Request:

*malloc(33)*

Worst Fit chooses

98

Remaining free blocks:

32    44    65

Second request:

*malloc(33)*

Largest block now:

65

Remaining:

32    44    32

|    |    |    |
|----|----|----|
| 32 | 44 | 32 |
|----|----|----|

This exact exercise appeared in previous exams.

## Comparison

| Algorithm | Fast   | Fragmentation |
|-----------|--------|---------------|
| First Fit | Yes    | Medium        |
| Best Fit  | No     | High          |
| Worst Fit | Medium | Medium        |

## Things to Remember

First Fit

First sufficiently large block.

Best Fit

Smallest possible block.

Worst Fit

Largest available block.

## Possible Exam Questions

A process has free blocks

32 44 98

Show the free list after two `malloc(33)` using Worst Fit.

Compare First Fit and Worst Fit.

Which algorithm searches the smallest usable block?

## 11 File Systems

A file system is responsible for storing and organizing files on secondary storage devices (HDD, SSD).

Its responsibilities include:

- storing files
- storing directories
- allocating disk blocks
- keeping metadata
- recovering free space

### 11.1 Linux Inode

Every file has an inode.

The inode stores metadata about the file, NOT the filename.

Typical information stored in an inode:

- Owner UID
- Group GID
- Permissions
- File size
- Creation / modification times
- Number of hard links
- Pointers to data blocks

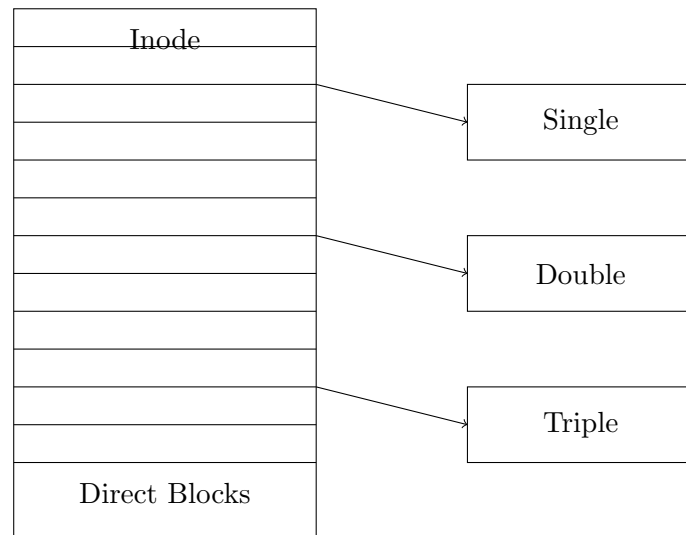
The filename is stored in the parent directory, which maps

$$filename \longrightarrow inode$$

### 11.2 Inode Structure

A Linux inode typically contains:

- 12 Direct pointers
- 1 Single Indirect pointer
- 1 Double Indirect pointer
- 1 Triple Indirect pointer



### 11.3 Maximum File Size

Suppose

$$BlockSize = B$$

Pointer size

$$Pointer = P$$

Pointers per indirect block:

$$N = \frac{B}{P}$$

Maximum file size:

$$12B + B \cdot N + B \cdot N^2 + B \cdot N^3$$

where

- first term = direct blocks
- second = single indirect
- third = double indirect
- fourth = triple indirect

This formula is frequently asked at exams.

### 11.4 Hard Links

A hard link is another directory entry pointing to the SAME inode.

“ file1 →+ — file2 →+ — inode — data “

Characteristics:

- Same inode
- Same data
- Same permissions
- Same owner

Deleting one filename does NOT delete the file.  
The inode is deleted only when

$$LinkCount = 0$$

## 11.5 Symbolic Links

A symbolic link is a small file containing the path of another file.

“ shortcut — V /path/file “

Characteristics:

- Different inode
- Contains pathname
- May point to another filesystem
- May become broken

## 11.6 Hard Link vs Symbolic Link

|                            | Hard Link | Symbolic Link |
|----------------------------|-----------|---------------|
| Same inode                 | Yes       | No            |
| May cross partitions       | No        | Yes           |
| Broken if original deleted | No        | Yes           |
| Stores pathname            | No        | Yes           |

## 11.7 Superblock

The superblock stores information about the entire filesystem.

Typical contents:

- Filesystem size
- Block size
- Number of free blocks
- Number of free inodes
- Filesystem type
- Location of inode table

Without the superblock the filesystem cannot normally be mounted.

## Possible Exam Questions

Explain the difference between Hard Links and Symbolic Links.  
Can a Hard Link point to another partition?  
What information is stored inside an inode?  
What information is stored inside the superblock?  
Derive the maximum file size formula.

## 12 RAID

RAID combines multiple disks into a single logical storage device.

Possible goals:

- Higher performance
- Fault tolerance
- Both

### 12.1 RAID 0

Striping only.

“ Disk1 A1 A3 A5 Disk2 A2 A4 A6 “

Advantages:

- Very fast
- Full capacity

Disadvantages:

- No redundancy
- One failed disk loses everything

### 12.2 RAID 1

Mirroring.

“ Disk1 A B C D Disk2 A B C D “

Advantages:

- Excellent reliability

Disadvantages:

- Uses twice the storage

### 12.3 RAID 5

Block striping with distributed parity.

Example:

“ D1 A1 A2 P

D2 A3 P A4

D3 P A5 A6 “

Advantages:

- Good performance
- Survives one disk failure

### 12.4 RAID 6

Like RAID 5 but stores TWO parity blocks.

Advantages:

- Survives two disk failures

Disadvantages:

- More parity overhead

## 12.5 RAID 10

RAID 1 + RAID 0.

Mirror first.

Stripe afterwards.

Advantages:

- Excellent performance
- Excellent redundancy

Disadvantages:

- Requires many disks

### Comparison

| RAID | Performance | Fault Tolerance |
|------|-------------|-----------------|
| 0    | Excellent   | None            |
| 1    | Medium      | Excellent       |
| 5    | High        | One Disk        |
| 6    | Medium      | Two Disks       |
| 10   | Excellent   | Excellent       |

### Possible Exam Questions

Compare RAID 0 and RAID 1.

Which RAID survives one failed disk?

Which RAID survives two failed disks?

Which RAID offers the highest performance?

## 13 Miscellaneous

### 13.1 Zombie Process

A zombie process has:

- finished execution
- released memory
- still has an entry in the process table

Reason:

The parent has not yet executed

*wait()*

Once the parent calls *wait()*, the zombie disappears.

### 13.2 Process vs Thread

|                   | Process   | Thread        |
|-------------------|-----------|---------------|
| Own address space | Yes       | No            |
| Own stack         | Yes       | Yes           |
| Communication     | IPC       | Shared memory |
| Creation cost     | High      | Low           |
| Context switch    | Expensive | Cheaper       |

### 13.3 Runtime of open/read

Example:

```
f=open("abc",ORDONLY);  
read(f,&c,1);  
close(f);
```

Behavior depends on the file type.

- Regular file  
Returns immediately.
- Pipe  
Blocks until data exists.
- FIFO  
Blocks until another process opens the other end.
- Terminal  
May wait for keyboard input.
- Device file  
Depends on the driver.

### Possible Exam Questions

What is a zombie process?  
Compare a process and a thread.  
Discuss the execution time of open/read depending on the file type.

## 14 Exam Cheat Sheet

- SJF = Non-preemptive
- STCF = Preemptive SJF
- Physical = Base + Logical
- Paging = Fixed Size
- Segmentation = Variable Size
- NRU removes Class 0 first
- Worst Fit chooses the largest free block
- Hard Links share the same inode
- Symbolic Links store a pathname
- RAID0 = Fast
- RAID1 = Mirror
- RAID5 = One failed disk
- RAID6 = Two failed disks
- RAID10 = RAID1 + RAID0
- Zombie = Finished but parent did not call wait()